



MotorDiagnosis

개발 및 Git 협업 표준

브랜치 · 커밋 · Pull Request · Python 개발 규칙

적용 저장소	ckdudwns/MotorDiagnosis
프로그램	Bind Edge AI 모터 센서 관제 프로토타입
문서 버전	1.0 · 2026-08-09
최종 병합자	@ckdudwns

01

문서 목적과 적용 원칙

이 문서는 팀의 기존 [파이썬] 개발 표준을 MotorDiagnosis의 실제 코드와 GitHub 협업 방식에 맞게 구체화한 실행 기준이다. 코드 스타일뿐 아니라 변경을 제안하고 검토하며 안전하게 main에 반영하는 전 과정을 다룬다.

한 줄 원칙

모든 변경은 작은 브랜치와 검증 가능한 PR로 제안하고, 최종 병합은 저장소 소유자 @ckdudwns만 수행한다.

규칙의 강도

등급	의미	예시
필수	예외 없이 적용한다. 예외가 필요하다면 PR에 이유와 대안을 기록한다.	직접 main 푸시 금지, 소유자 병합, 비밀번호 금지
권장	특별한 이유가 없다면 적용한다. 다른 선택은 PR에서 설명한다.	Draft PR 조기 공유, 88자 Black 포맷
선택	상황에 따라 도입하며 팀 합의 후 필수로 승격할 수 있다.	Ruff, 추가 정적 분석, PyTorch

현재 프로그램을 기준으로 한 핵심 판단

현재 구현	표준에 반영한 결정
단일 app.py + ThreadingHTTPServer	다음 기능부터 HTTP, 서비스, 데이터 모델 분리를 우선하며 공유 상태 갱신은 동시성을 검토한다.
GET/POST API와 HTML이 한 파일에 공존	API 스키마 변경과 UI 변경을 PR 본문에서 별도 영향 범위로 표시한다.
JSON은 camelCase, Python-CSV는 snake_case	외부 JSON 호환성은 유지하고 Python 내부와 CSV 헤더는 snake_case를 사용한다.
진동·음향·RPM·이상 점수 텔레메트리	센서 단위와 시간 기준을 이름 또는 스키마에 명시하고 임계값 변경은 데이터 영향 PR로 분류한다.
CSV UTF-8 BOM 제공	Excel 호환성을 회귀 검증 항목으로 유지한다.

목차

구분	내용
Git 협업	브랜치, 커밋, PR, 리뷰와 병합
Python	환경, 포맷, 명명, 타입과 문서화
MotorDiagnosis	API, 센서 데이터, 이벤트, 동시성, 로깅
품질	테스트, 보안, 체크리스트, 운영

02

브랜치 규칙

브랜치는 작업의 종류와 목적이 이름만으로 보이도록 만든다. 하나의 브랜치에는 하나의 목적만 담고, 항상 최신 main 에서 시작한다.

이름 형식

```
<type>/<short-kebab-description>
```

예: feat/telemetry-history

예: fix/event-review-404

접두사	사용 시점	프로젝트 예시
feat	사용자가 확인할 수 있는 새 기능	feat/csv-date-range
fix	잘못된 동작 또는 회귀 수정	fix/invalid-site-response
refactor	동작을 바꾸지 않는 구조 개선	refactor/split-http-handler
test	테스트 추가·수정	test/telemetry-schema
docs	문서만 변경	docs/python-standard
chore	설정·의존성·정리	chore/black-config
ci	자동 검사·배포 흐름	ci/pr-quality-checks
hotfix	운영 장애의 긴급 수정	hotfix/export-encoding

작성 세부 규칙

- 영문 소문자, 숫자, 하이픈만 사용한다. 공백·한글·밑줄은 사용하지 않는다.
- 설명은 2~5개 단어로 짧게 작성하고 update, work 같은 모호한 표현을 피한다.
- 이슈 번호가 있으면 뒤에 붙일 수 있다: fix/event-review-404-#27.
- 브랜치는 PR 병합 후 삭제한다. 장기 브랜치는 예외이며 책임자와 동기화 방식을 PR 또는 이슈에 기록한다.

기본 작업 흐름

- 1 main을 최신 상태로 맞춘다.
- 2 규칙에 맞는 작업 브랜치를 만든다.
- 3 검토 가능한 작은 커밋으로 작업한다.
- 4 진행 공유가 필요하면 Draft PR을 먼저 연다.
- 5 테스트·자체 검토 후 Ready for review로 전환한다.
- 6 @ckdudwns의 검토와 최종 병합을 기다린다.

금지

main 직접 푸시, 강제 푸시, 리뷰 우회를 금지한다. 긴급 수정도 가능한 한 PR을 만들고, 불가피한 예외는 즉시 사후 기록한다.

03

커밋 메시지 규칙

커밋 메시지는 Conventional Commits 1.0.0 형식을 따른다. Squash merge를 사용하므로 개별 커밋뿐 아니라 **최종 PR 제목도 같은 형식**으로 작성한다.

기본 형식

```
<type>(<optional-scope>): <summary>  
  
<optional body: 왜 변경했는지, 고려한 대안>  
  
<optional footer: Refs/Closes/BREAKING CHANGE>
```

사용 타입

type	의미	예시
feat	기능 추가	feat(telemetry): 진동 이력 조회 추가
fix	버그 수정	fix(api): 없는 이벤트에 404 반환
refactor	동작 없는 구조 변경	refactor(server): 응답 헬퍼 분리
test	테스트 변경	test(export): CSV 헤더 검증 추가
docs	문서 변경	docs: 로컬 실행 절차 보완
chore	설정·의존성·정리	chore: Python 3.12 설정 추가
ci	자동화 변경	ci: PR 포맷 검사 추가
perf	성능 개선	perf(telemetry): 중복 자산 조회 제거
revert	변경 되돌림	revert: 이벤트 정렬 변경 취소

권장 scope

scope	범위	관련 코드
api	HTTP 라우팅·응답	AppHandler, do_GET, do_POST
telemetry	센서 시계열 생성·조회	telemetry_for
anomaly	이상 점수·주입	inject_anomaly, 임계값
event	이벤트 조회·검토	review_event, EVENTS
ui	대시보드 HTML/CSS/JS	render_page
export	CSV 스키마·인코딩	send_csv
config	PORT·파라미터·실행 환경	HOST, PORT, PARAMETERS

요약 문장 규칙

- 무엇이 바뀌었는지를 한 문장으로 쓰고 마침표는 생략한다.
- '수정', '업데이트'만 쓰지 말고 대상과 결과를 포함한다.

- API·CSV 스키마가 호환되지 않게 바뀌면 ! 또는 BREAKING CHANGE:를 사용한다.
- 서로 독립적인 기능·리팩터링·포맷 변경은 가능한 한 별도 커밋으로 나눈다.

좋은 예

fix(export): CSV 진동 컬럼에 RMS 단위 명시
feat(event): 검토 완료 시각을 ISO 8601로 저장

피해야 할 예

fix, update app, 처럼 검토자가 영향 범위를 알 수 없는 메시지

04

Pull Request 작성·리뷰·병합

PR은 변경을 설명하는 검토 단위이자, Squash merge 후 남는 최종 변경 기록이다. 제목은 커밋 규칙과 동일하고 본문은 검증 가능해야 한다.

제목

feat(telemetry): 설비별 센서 이력 조회 추가
fix(ui): 이벤트 미선택 상태에서 저장 차단

본문 필수 항목

항목	작성 내용
변경 목적	왜 필요한지, 어떤 사용자·운영 문제를 해결하는지
주요 변경	기능·API·UI·데이터·설정 단위의 핵심 차이
영향 범위	API 경로, JSON/CSV 스키마, 임계값, UI, 운영 설정
확인 방법	실행 명령, 요청 예시, 기대 결과, 화면 변경 증거
관련 이슈	Closes #번호 또는 없음
위험과 복구	데이터 호환성, 롤백 방법, 알려진 제한

MotorDiagnosis PR 체크리스트

- **API:** 정상·오류 상태 코드, content-type, 빈 요청, 잘못된 ID를 확인했다.
- **텔레메트리:** site_id/asset_id, 시간 기준, 진동·음향·RPM·점수 단위를 확인했다.
- **이벤트:** 설비 이상과 센서 이상을 구분하고 검토 라벨 전이를 확인했다.
- **CSV:** 컬럼 순서, 단위, UTF-8 BOM, Excel 열기 결과를 확인했다.
- **UI:** 900px 이하 반응형, 빈 상태, 오류 상태, 한국어 문자열을 확인했다.
- **보안:** 비밀번호·개인정보·원시 운영 데이터·로컬 파일이 포함되지 않았다.

리뷰와 병합 정책

단계	필수 조건
Draft	작업 중 공유 가능. 병합 대상이 아니며 미완료 항목을 명시한다.
Ready for review	자체 검토, 테스트, 문서 업데이트를 완료한다.
Review	@ckdudwns의 승인, 모든 대화 해결, 검사 통과가 필요하다.
Merge	@ckdudwns만 Squash and merge를 수행한다.
After merge	원격 브랜치를 삭제하고 필요 시 이슈·변경 로그를 닫는다.

GitHub 강제 설정

main Ruleset에 **Require a pull request, Restrict updates**, force push·삭제 차단을 켜고, bypass는 Repository admins 만 허용한다. 다른 협업자는 Admin이 아닌 Write 이하로 둔다.

CODEOWNERS

현재 .github/CODEOWNERS의 전체 기본 검토자는 @ckdudwns다. Ruleset에서 Code Owner review를 요구해야 승인 조건이 실제 병합 차단으로 작동한다.

05

Python 개발 환경

팀 공유 표준을 따라 Python 3.12 계열을 기준으로 한다. 재현 가능한 로컬 환경을 만들고, 외부 의존성은 실제로 필요한 경우에만 추가한다.

항목	표준
Python	기준 버전 3.12.4, 허용 범위 3.12.x
가상환경	저장소 루트의 .venv 사용
포맷터	Black 필수, 기본 line length 88
IDE	VS Code 권장, 다른 IDE도 동일 포맷 결과면 허용
의존성	현재 app.py는 표준 라이브러리만 사용. 새 외부 패키지는 requirements.txt에 직접 의존성으로 기록
PyTorch	실제 모델 추론이 필요한 경우에만 하드웨어·CUDA 조합을 확인하여 별도 설치

초기 설정

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS / Linux
source .venv/bin/activate
# 의존성이 생긴 경우
python -m pip install -r requirements.txt
```

포맷과 기본 검사

```
python -m black .  
python -m black --check .  
python -m compileall app.py
```

현재 프로젝트에 대한 적용

현재 실행 경로는 `python app.py`이고 외부 런타임 패키지가 없다. 단순히 환경을 복제하기 위해 전체 환경의 `pip freeze` 결과를 넣지 말고, 실제 직접 의존성만 관리한다.

의존성 추가 PR에 포함할 내용

- 패키지가 해결하는 문제와 표준 라이브러리 대안 검토 결과
- 지원 Python 버전과 운영 환경 호환성
- 고정 또는 호환 버전 범위와 라이선스
- 설치·실행·제거 방법
- 모델 또는 대용량 파일을 저장소에 직접 포함하지 않는 배포 방식

06

Python 코드 스타일과 명명

PEP 8을 기본으로 하고 Black 결과를 포맷의 단일 기준으로 삼는다. 기존 공유 표준의 PascalCase·snake_case 원칙을 유지하되, 센서 데이터는 단위와 의미를 이름에 명확히 남긴다.

명명 규칙

대상	형식	MotorDiagnosis 예시
클래스	PascalCase	AppHandler, TelemetryService, EventRepository
함수·변수	snake_case	telemetry_for, asset_id, event_count
상수	UPPER_SNAKE_CASE	HOST, PORT, ANOMALY_SCORE_THRESHOLD
모듈·폴더	짧은 소문자 snake_case	telemetry.py, event_store.py
불리언	is/has/can 접두사	is_online, has_open_event, can_export
식별자	대상 + _id	site_id, asset_id, event_id

센서·통계 이름

의미	권장 이름	비고
진동 RMS	vibration_rms_mm_s	가능하면 물리 단위를 포함
음압	acoustic_db	dB 기준이 다르면 메타데이터에 명시
회전수	rpm	도메인에서 확립된 약어는 허용
이상 점수	anomaly_score	범위 0~100을 스키마에 기록
수집 주기	telemetry_interval_sec	시간 단위를 접미사로 명시
평균·표준편차	vibration_mean, vibration_std	mean, max, min, std 등의 접미사 사용

함수 설계

- 함수명은 동사 또는 명확한 기능으로 시작한다: `get_site`, `build_telemetry_points`, `export_telemetry_csv`.
- 입력과 반환 타입을 타입 힌트로 명시하고, API 경계에서는 검증된 데이터만 서비스 함수에 전달한다.
- 하나의 함수가 HTML 생성, 데이터 변경, HTTP 응답을 동시에 담당하지 않도록 분리한다.
- 가져오기와 직접 실행을 구분해야 하는 모듈은 반드시 `if __name__ == "__main__":`을 사용한다.
- 예상 가능한 오류는 구체적인 예외로 처리하고 `bare except:`는 사용하지 않는다.

외부 스키마와 내부 이름

호환성 원칙

현재 브라우저 클라이언트가 사용하는 JSON 키 `siteId`, `assetId`, `onlineDevices`는 외부 계약이다. Python 내부 변수는 `site_id`처럼 쓰고, 키 형식 변경은 별도의 breaking-change PR에서 클라이언트와 함께 변경한다.

MotorDiagnosis 도메인·API 표준

모터 진단 데이터는 단순한 숫자 목록이 아니라 시간, 설비, 단위, 수집 상태가 결합된 기록이다. API와 CSV 모두 같은 의미를 유지해야 한다.

API 응답 원칙

상황	기준
성공 조회	200, application/json; charset=utf-8
생성	201, 생성된 이벤트 반환
잘못된 입력	400, 안전한 오류 코드와 설명
없는 리소스	404, 잘못된 ID를 첫 항목으로 대체하지 않음
지원하지 않는 메서드	405
예상하지 못한 오류	500, 내부 예외·경로·비밀정보 미노출

텔레메트리 스키마

필드	의미	검증
timestamp	측정 시각	ISO 8601 UTC 권장, 시간대 포함
site_id / asset_id	발전소·설비 식별자	존재 여부 확인, 임의 기본값 대체 금지
vibration_rms_mm_s	진동 RMS	유효한 수, 센서 범위 검증
acoustic_db	음압 수준	측정 기준과 범위 기록
rpm	회전수	0 이상, 비가동 상태 별도 표현
anomaly_score	이상 점수	0~100, 모델·임계값 버전 추적

이벤트와 진단 표현

- 설비 이상 후보, 센서 이상, 통신 장애를 서로 다른 severity 또는 event_type으로 구분한다.
- 자동 점수만으로 고장을 확정하지 않는다. 사용자 화면에서는 '이상 후보'와 검토 상태를 분리한다.
- 검토 라벨은 **확인 필요** → **정상/오탐** | **이상 확인** → **정비 완료** 흐름을 기본으로 하며 변경 시 이력과 시각을 남긴다.
- 임계값 변경은 결과 분포와 과거 이벤트에 미치는 영향을 PR에 기록한다.

CSV 내보내기

- 헤더는 snake_case와 단위를 사용한다: vibration_rms_mm_s, acoustic_db.
- Excel 호환을 위해 현재 UTF-8 BOM 동작을 유지하며 회귀 테스트한다.
- 컬럼 삭제·이름 변경·단위 변경은 breaking change다. 새 컬럼 추가도 소비자 영향을 PR에 명시한다.
- 파일명은 <site_id>_<asset_id>_<date>.csv 형식을 권장한다.

08

구조·동시성·오류·로그

현재 코드는 빠른 검증에 적합한 단일 파일 프로토타입이다. 이후 변경은 기능을 추가하는 동시에 책임 경계를 명확히 하는 방향으로 진행한다.

권장 구조

```
motor_diagnosis/
server.py # HTTP 라우팅과 응답
services/telemetry.py # 텔레메트리 생성·조회
services/events.py # 이상 이벤트 검토
models.py # 타입과 스키마
settings.py # 환경변수·임계값
web/ # HTML/CSS/JS
tests/ # 단위·API·스키마 테스트
```

분리 기준

- 새 API가 추가되면 라우팅 분기와 도메인 로직을 같은 함수에 넣지 않는다.
- 새 UI 기능은 거대한 `render_page()` 문자열에 계속 누적하지 않고 정적 자산 또는 템플릿으로 분리한다.
- 전역 SITES, EVENTS를 직접 변경하는 새 코드는 추가하지 않고 저장소·서비스 계층을 거친다.
- 구조 리팩터링과 기능 변경은 가능한 한 서로 다른 PR로 나눈다.

ThreadingHTTPServer 동시성

현재 위험

여러 요청이 동시에 `EVENTS.insert()` 또는 사이트 상태를 갱신할 수 있다. 프로토타입을 넘어 실제 동시 사용을 허용하기 전에는 lock, 단일 writer, 데이터베이스 트랜잭션 중 하나를 적용해야 한다.

오류 처리

원칙	적용
Fail clearly	없는 <code>site_id</code> 를 <code>SITES[0]</code> 으로 대체하지 않고 명시적인 오류로 반환
Validate at boundary	JSON 형식, Content-Length, 필수 필드, 허용 라벨 검증
No secret leakage	응답에 traceback, 로컬 경로, 환경변수 값을 포함하지 않음
Keep context	로그에는 <code>request_id</code> , <code>endpoint</code> , <code>site_id/asset_id</code> 를 구조화해 기록

로그

- 운영 코드에서는 `print` 대신 표준 logging을 사용한다.
- `log_message()`를 완전히 무시하지 말고 접근 로그를 INFO, 입력 오류를 WARNING, 예외를 ERROR로 남긴다.
- 센서 원시값·개인정보·비밀정보를 로그에 남기지 않는다. 장비 ID도 필요 시 마스킹 정책을 적용한다.

09

테스트·품질·보안

PR의 검증 수준은 변경 위험과 비례해야 한다. 현재 테스트가 적더라도 새 기능은 최소한 재현 가능한 확인 절차와 함께 제출한다.

기본 검사

검사	최소 기준
문법	python -m compileall app.py 통과
포맷	python -m black --check . 통과
실행	python app.py 후 /api/health가 200과 ok=true 반환
API	변경 endpoint의 정상 400-404 경로 확인
CSV	헤더, 행 수, BOM, Excel 호환 확인
UI	주요 흐름과 900px 이하 화면 확인

변경별 필수 테스트

변경 종류	테스트 초점
telemetry	동일 site/asset 입력의 재현성, 값 범위, 72개 포인트
anomaly/event	상태 전이, 이벤트 수 증가, 잘못된 ID와 라벨
HTTP	상태 코드, 헤더, JSON 인코딩, 빈 body
export	스키마·단위·파일명·BOM
UI	로딩·빈 상태·오류 표시·키보드 접근
concurrency	동시 요청 시 ID 중복·유일·부분 갱신 없음

보안과 저장소 위생

- 비밀번호, 토큰, 비밀번호, 인증서, 실제 장비 접속정보는 커밋하지 않는다.
- 실제 원시 센서 데이터와 내부 기획 문서는 공개 범위를 확인하기 전 저장소에 추가하지 않는다.
- 생성된 CSV, 로그, 캐시, 로컬 DB, 가상환경은 .gitignore로 제외한다.
- 파일 업로드 또는 경로 입력을 추가하면 크기·확장자·경로 순회 공격을 검증한다.
- 외부 입력을 HTML에 삽입할 때 escape 처리하여 XSS를 방지한다.

완료 정의

코드가 동작하는 것만으로 완료가 아니다. 포맷, 테스트, 오류 경로, 문서, 스키마 호환성, 보안 확인까지 끝나야 Ready for review다.

10

빠른 참조

작업 시작부터 병합까지 이 한 페이지를 체크리스트로 사용한다.

작업 시작

- 최신 main에서 <type>/<kebab-description> 브랜치를 만든다.
- 하나의 목적과 완료 조건을 정한다.
- API·데이터·UI·운영 중 영향 범위를 먼저 표시한다.

커밋

```
feat(telemetry): 발전소별 센서 이력 추가
fix(api): 존재하지 않는 설비에 404 반환
test(export): CSV BOM 회귀 테스트 추가
```

PR 본문 예시

```
## 변경 목적
CSV 소비자가 진동 값의 물리 단위를 구분할 수 있도록 한다.

## 주요 변경
- vibration_rms 컬럼을 vibration_rms_mm_s로 명확화
- 기존 소비자를 위한 마이그레이션 안내 추가

## 확인 방법
- /api/export 응답의 헤더·BOM·행 수 확인
- Excel에서 한글과 숫자 열림 확인

## 영향 및 복구
- CSV 스키마 breaking change
- 문제 발생 시 이전 헤더로 되돌림
```

병합 전

- PR 제목이 Conventional Commit 형식이다.
- 자체 검토와 필수 테스트가 끝났다.
- 비밀정보·원시 데이터·생성물이 없다.
- 모든 리뷰 대화와 검사 항목이 해결됐다.
- @ckdudwns가 최종 Squash and merge를 수행한다.

기억할 경계

Python 내부는 snake_case, 클래스는 PascalCase, 상수는 UPPER_SNAKE_CASE. 기존 외부 JSON camelCase는 호환성 계약이므로 임의로 바꾸지 않는다.

운영.개정.참고 자료

이 표준은 팀의 기본 규칙이다. 예외는 숨기지 않고 PR에 이유, 영향, 종료 조건을 기록한다. 규칙 자체의 변경도 docs 브랜치와 PR을 통해 검토한다.

문서 운영

항목	기준
소유자	저장소 소유자 @ckdudwns
개정 방식	docs/ 브랜치 → PR → 소유자 병합
충돌 우선순위	보안·데이터 무결성 → 저장소 규칙 → 본 문서 → 일반 권장사항
예외 기록	사유, 기간, 영향, 후속 이슈, 복구 계획
점검 시점	프로젝트 구조 변경, 첫 운영 배포, 모델 도입, API 버전 변경 시

근거 자료

- 팀 공유 문서: [\[파이썬\] 개발 표준](#) - Python 3.12.x, venv, Black, 명명법, main guard
- 로컬 저장소: app.py, CONTRIBUTING.md, .github/CODEOWNERS, .github/pull_request_template.md
- Python: [PEP 8 - Style Guide for Python Code](#)
- Black: [The Black Code Style](#)
- Commit: [Conventional Commits 1.0.0](#)
- GitHub: [Managing and standardizing pull requests](#)
- GitHub: [Available rules for rulesets](#)

적용 권고

현재 로컬에 마련된 CONTRIBUTING, PR 템플릿, CODEOWNERS와 이 PDF의 핵심 규칙을 함께 커밋하고, GitHub Ruleset을 활성화하면 문서 규칙과 실제 병합 권한이 일치한다.

END OF STANDARD